

# Übungsblatt 4

---

## HTTP und REST

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

# Lernziele

- **HTTP-Methoden, Status-Codes** und **Header** korrekt anwenden
- **Caching-Strategien** mit **Cache-Control**, **ETag** und **Last-Modified** verstehen
- **RESTful URL-Design** und Ressourcenhierarchien entwerfen
- **HATEOAS** und den Reifegrad nach Richardson einordnen
- **REST vs. SOAP** vergleichen und **OpenAPI**-Spezifikationen lesen
- **TLS-Handshake** und Sicherheitsgrundlagen für **HTTPS** kennen

# Modul A — HTTP

---

# Übung 4.A.1: HTTP-Methoden und Status-Codes

Ordnen Sie jeder Situation die korrekte HTTP-Methode und den passenden Status-Code zu:

| Situation                                | Methode | Status-Code |
|------------------------------------------|---------|-------------|
| Produktliste abrufen                     | ?       | ?           |
| Neues Produkt anlegen                    | ?       | ?           |
| Produktname aktualisieren (nur ein Feld) | ?       | ?           |
| Produkt vollständig ersetzen             | ?       | ?           |
| Produkt löschen                          | ?       | ?           |
| Ressource nicht gefunden                 | —       | ?           |
| Validierungsfehler im Request-Body       | —       | ?           |
| Server-interner Fehler                   | —       | ?           |

# Übung 4.A.2: Caching-Strategien

Ein Online-Shop liefert verschiedene Ressourcen. Bestimmen Sie für jede die geeignete Caching-Strategie:

| Ressource                                                                  | Strategie (kein Cache / kurz / lang / ewig) | HTTP-Header |
|----------------------------------------------------------------------------|---------------------------------------------|-------------|
| Produktbild <code>logo.png</code> (unveränderlich, mit Hash im Dateinamen) | ?                                           | ?           |
| Produktliste (aktualisiert sich stündlich)                                 | ?                                           | ?           |
| Warenkorb-Inhalt (nutzerspezifisch)                                        | ?                                           | ?           |
| JavaScript-Bundle <code>app.abc123.js</code> (Hash im Namen)               | ?                                           | ?           |

# Modul B — REST API Design

---

# Übung 4.B.1: RESTful URL-Design

Ein Fitness-App-Backend soll folgende Operationen unterstützen. Entwerfen Sie RESTful URLs:

1. Alle Trainingspläne eines Nutzers abrufen
2. Einen neuen Trainingsplan für einen Nutzer erstellen
3. Eine spezifische Übung aus einem Trainingsplan abrufen
4. Eine Übung in einem Trainingsplan aktualisieren
5. Alle abgeschlossenen Trainingseinheiten eines Nutzers in einem Zeitraum filtern

Welche URL-Muster sind dabei problematisch: `/getTrainings`, `/user/deleteById?id=5?`

# Übung 4.B.2: HATEOAS und API-Versionierung

1. Was bedeutet **HATEOAS** und welchen praktischen Vorteil bietet es gegenüber einer reinen Daten-API?  
Zeigen Sie ein JSON-Beispiel für eine Produktressource mit HATEOAS-Links.
2. Nennen Sie zwei Strategien für **API-Versionierung** und diskutieren Sie je einen Vorteil und Nachteil.

# Modul C — REST vs. SOAP

---

# Übung 4.C.1: REST vs. SOAP Entscheidung

Entscheiden Sie für jedes Szenario, ob REST oder SOAP besser geeignet ist, und begründen Sie:

| Szenario                                                        | REST oder SOAP? | Begründung |
|-----------------------------------------------------------------|-----------------|------------|
| Mobile App ruft Produktdaten ab                                 | ?               | ?          |
| Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen) | ?               | ?          |
| IoT-Sensoren senden Messwerte (geringe Bandbreite)              | ?               | ?          |
| Legacy-Enterprise-Integration mit strenger Vertragsbindung      | ?               | ?          |

# Modul D — Sicherheit und API-Dokumentation

---

# Übung 4.D.1: TLS/SSL — Transport-Sicherheit

1. Was war das zentrale Sicherheitsproblem von **SSL 2.0**, das zur Entwicklung von TLS führte?
2. Ist TLS heute für Web-Anwendungen verpflichtend oder optional? Begründen Sie aus technischer und regulatorischer Sicht.
3. Erklären Sie den **TLS-Handshake** in 4 Schritten: Was wird ausgetauscht und welches Ziel hat jeder Schritt?
4. Was bedeutet **Certificate Pinning** und für welche Art von Anwendung ist es besonders relevant?

# Übung 4.D.2: Swagger/OpenAPI — API-Dokumentation

1. Was ist **OpenAPI** (früher: Swagger) und welches Problem löst es bei der Entwicklung von REST-APIs?
2. Was bedeutet **Design-First** vs. **Code-First** bei REST-APIs? Nennen Sie je einen Vorteil.
3. Gegeben: eine OpenAPI-Spezifikation beschreibt folgenden Endpunkt:

```
/products/{id}:  
  get:  
    summary: Get a product by ID  
    parameters:  
      - name: id  
        in: path  
        required: true  
        schema:  
          type: integer  
    responses:
```



```
'200':  
  description: Product found  
'404':  
  description: Product not found
```

a) Welchen HTTP-Status liefert der Endpunkt bei einer nicht vorhandenen Produkt-ID? b) Warum ist `id` als `in: path` (nicht `in: query`) definiert?